

程序设计训练 (Rust 语言)



第 3 讲：标准库

韩文弢

清华大学

2025—2026 学年度夏季学期

本讲内容

3.1 字符串	2
3.2 Option 枚举类型	12
3.3 错误处理	25
3.4 容器	38
3.5 迭代器	52
3.6 自动测试	69
3.7 小结	77

3.1 字符串

3.1.1 字符串类型

- Rust 的字符串处理机制比较复杂。
 - 主要是用 UTF-8 编码的 Unicode 字符序列。
 - 不是空字符 `'\0'` 结尾的 C 风格字符串，可以包含空字符。
- 主要有两大类：`&str` 和 `String`。

3.1.2 字符编码

- 编码：字符在计算机内部的表示方式
- 早期：ASCII 码，以英文字符为主，7 位二进制
- 中文：GB 2312-1980 《信息交换用汉字编码字符集》，6,763 个汉字，两个字节
 - GB 18030-2005 《信息技术中文编码字符集》，70,244 个汉字，两个字节或四个字节
- Unicode：试图把全世界的文字都纳入进来，Unicode 15.1 包含 161 种文字的 149,813 个字符，四个字节
 - 常用 UTF-8 的形式来表示，变长一到四个字节

Unicode 中文乱码速查表

xxxxxx	示例	特点	产生原因
古文码	鑒辨溪瑕佻ソ濂藉滂豺お澶十悝涓?	大都为不认识的古文，并加杂日韩文	以 GBK 方式读取 UTF-8 编码的中文
口字码	???"Ç?2?AX????	大部分字符为小方块	以 UTF-8 的方式读取 GBK 编码的中文
符号码	ç"±æœ`è ?å ¥ ½å ¥ ½ä- ä1 å¤©å¤©å¤'ä,Š	大部分字符为各种符号	以 ISO8859-1 方式读取 UTF-8 编码的中文
拼音码	óéÔÂàaoÃoÃÑģĩ°iiiiöéĩ	大部分字符为头顶带有各种类似声调符号的字母	以 ISO8859-1 方式读取 GBK 编码的中文
问句码	由月要好好学习天天向??	字符串长度为偶数时正确，长度为奇数时最后的字符变为问号	以 GBK 方式读取 UTF-8 编码的中文，然后又用 UTF-8 的格式再次读取
锟拷码	锟斤拷锟斤拷要锟矫猴拷学习锟斤拷锟斤拷锟斤拷	全中文字符，且大部分字符为“锟斤拷”这几个字符	以 UTF-8 方式读取 GBK 编码的中文，然后又用 GBK 的格式再次读取
烫烫烫	烫烫烫烫烫烫烫烫烫烫烫烫烫烫烫烫烫烫	字符显示为“烫烫烫”这几个字符	VC Debug 模式下，栈内存未初始化
屯屯屯	屯屯屯屯屯屯屯屯屯屯屯屯屯屯屯屯屯屯	字符显示为“屯屯屯”这几个字符	VC Debug 模式下，堆内存未初始化

出自 <https://github.com/justjavac/unicode-encoding-error-table>。

- &str 是字符串切片，是切片的一种。
- 形如 "string literals" 的字符串字面值是 &str 类型的¹。
- **不能**用方括号来做形如 some_str[i] 的索引，因为每个 Unicode 字符可能有多个字节。
- 正确的做法是在 chars() 中迭代：

```
1 for c in "1234".chars() {  
2     // ...  
3 }
```



¹更准确地说，是 &'static str 类型，在讲生命周期时会进一步解释。

3.1.5 String

- String 是分配在堆上的，可以动态增长。
 - 和 Vec 类似，实际上就是在 Vec<u8> 外面包了一层。
- 也不能用下标来索引。
 - 可以通过 s.chars().nth(i) 来访问某个字符。
- 通过取引用的方式可以获得 &str。

```
1 let s0: String = String::new();  
2 let s1: String = "foo".to_string();  
3 let s2: String = String::from("bar");  
4 let and_s: &str = &s0;
```



3.1.6 str

- 如果 `&str` 是一种字符串类型，那么 `str` 到底是什么？
- `str` 是一种编译时大小未知的类型。
 - 不能直接绑定 `str`，只能通过引用的形式来使用。

3.1.7 字符串连接操作

- 可以用 + 连接一个 String 和一个 &str 类型的字符串（注意顺序）：

```
1 let a = String::from("hello");  
2 let b = String::from(" world");  
3 let c = a + &b;  
4 // `a` is moved and can no longer be used here.
```

 Rust

- 如果想保留第一个 String，需要做一份克隆 (clone)：

```
1 let a = String::from("hello");  
2 let b = String::from(" world");  
3 let c = a.clone() + &b;  
4 // `a` is still valid here.
```

 Rust

3.1.8 字符串连接操作和实现

- 如果要连接两个 `&str`，需要把第一个转换成 `String`：

```
1 let a = "hello";  
2 let b = " world";  
3 let c = a.to_string() + b;
```

 Rust

- 连接操作的实现代码：

```
1 fn add(mut self, other: &str) -> String {  
2     self.push_str(other);  
3     self  
4 }
```

 Rust

- 当然，也可以使用宏 `format!("{a}{b}")`

3.1.9 String 与 &str 并存的设计原因

- `&str` 能够提供 `String` 的一个视图，正如切片之于数组或 `Vec` 向量那样。
- 拷贝 `String` 的代价比较大，而且借用的时候并不一定需要整个字符串。
- `&str` 提供了一种低开销传递部分 `String` 字符串内容的方法，而且节约内存。
- 一般而言，如果要处理字符串，通常会使用 `String`，同时可以用 `&str` 来借用其中的内容。

3.2 Option 枚举类型

```
1 enum Option<T> {  
2     None,  
3     Some(T),  
4 }  
5  
6 use Option::{None, Some};
```



- Option<T> 是一个枚举类型，同时也是泛型类型。
- 为某种已有类型提供了表示**没有**或者**空值**的概念。
- 在 Rust 中，在需要返回空值时，推荐使用 Option<T>。
 - 而不是返回诸如 NaN、-1、null 等特殊值。
- 类型 T 可以是任何类型，没有限制。

3.2.2 Option::unwrap()

- 在处理 Option 类型的数据时，一定会面对 None 的情况，忽略 None 是一种很常见的处理方法。

```
1 // fn foo() -> Option<i32>
2
3 match foo() {
4     None => (),
5     Some(value) => {
6         bar(value)
7         // ...
8     },
9 }
```



3.2.3 Option::unwrap()

```
1 fn unwrap<T>(self) -> T {  
2     match self {  
3         None => panic!("called `Option::unwrap()` on a `None` value"),  
4         Some(value) => value,  
5     }  
6 }  
7  
8 let x = foo().unwrap();  
9 let y = bar(x);  
10 // ...
```



Rust

- Option::unwrap() 在遇到 None 时会恐慌并输出固定的内容。
- 更好的做法是调用 expect(self, msg: &str) -> T。
 - 它可以在遇到 None 时以指定的信息执行恐慌。

3.2.4 Option::map()

- 如果希望对一个 Option 进行变换，也就是有值的时候作用一个函数，空值的时候继续保持空值，可以调用：

```
1 fn map<U, F>(self, f: F) -> Option<U>
2     where F: FnOnce(T) -> U {
3     match self {
4         None => None,
5         Some(x) => Some(f(x))
6     }
7 }
8
9 // fn foo() -> Option<i32>
10
11 let x = foo().map(|x| bar(x));
```



Rust

- 类似的还有 and_then:

```
1  fn and_then<U, F>(self, f: F) -> Option<U>
2      where F: FnOnce(T) -> Option<U> {
3      match self {
4          Some(x) => f(x),
5          None => None,
6      }
7  }
8
9  // fn foo() -> Option<i32>
10
11 let x = foo().and_then(|x| Some(bar(x)));
```



- 注意，和 map 相比，f 的类型从 $T \rightarrow U$ 变为 $T \rightarrow \text{Option}(U)$ 。

3.2.6 Option::unwrap_or()

- 如果对于空值的情况有合理的默认值，可以用 `unwrap_or` 提供。

```
1  impl<T> Option<T> {  
2      fn unwrap_or<T>(self, default: T) -> T {  
3          match self {  
4              None => default,  
5              Some(value) => value,  
6          }  
7      }  
8  }  
9  
10 // fn foo() -> Option<i32>  
11  
12 let x = foo().unwrap_or(42);
```



Rust

3.2.7 Option::unwrap_or_else()

- 如果默认值由闭包计算而来，则使用 `unwrap_or_else`。

```
1  impl<T> Option<T> {  
2      fn unwrap_or_else<T>(self, f: F) -> T  
3          where F: FnOnce() -> T {  
4          match self {  
5              None => f(),  
6              Some(value) => value,  
7          }  
8      }  
9  }  
10  
11 // fn foo() -> Option<i32>  
12  
13 let x = foo().unwrap_or_else(|| a + b);
```



Rust

3.2.8 其他方法

- `fn is_some(&self) -> bool`
- `fn is_none(&self) -> bool`
- `fn map_or<U, F>(self, default: U, f: F) -> U`
 - `where F: FnOnce(T) -> U`
 - U 类型的默认值
- `fn map_or_else<U, D, F>(self, default: D, f: F) -> U`
 - `where D: FnOnce() -> U, F: FnOnce(T) -> U`
 - D 类型的闭包用于计算默认值

3.2.9 其他方法 (续)

- `fn ok_or(self, err: E) -> Result<T, E>`
- `fn ok_or_else(self, default: F) -> Result<T, E>`
 - `where F: FnOnce() -> E`
 - 与 `unwrap_or` 相似，用于错误处理。
- `fn and<U>(self, optb: Option<U>) -> Option<U>`
 - 如果 `self` 是 `None`，则返回 `None`，否则返回 `optb`。
- `fn or(self, optb: Option<T>) -> Option<T>`
 - 如果 `self` 是 `Some(_)`，则返回 `self`，否则返回 `optb`。
- `fn xor(self, optb: Option<T>) -> Option<T>`
 - 如果 `self` 和 `optb` 恰好有一个是 `Some(_)`，则返回这个 `Some`，否则返回 `None`。

3.2.10 搜索需要的方法

- Rust 文档可以通过函数的参数和返回值的类型搜索函数
 - 例如搜索 `Option<T> -> Option<U>`
 - 可以得到 Option 的 `and`、`zip`、`map`、`unzip`、`and_then`、`zip_with` 等方法
- 也可以参考网站 <https://jethrogb.github.io/rust-combinators/>

3.2.11 Option 例子

3.2 Option 枚举类型

```
1 fn divide(numerator: f64, denominator: f64) -> Option<f64> {
2     if denominator == 0.0 {
3         None
4     } else {
5         Some(numerator / denominator)
6     }
7 }
8
9 // The return value of the function is an option
10 let result = divide(2.0, 3.0);
11
12 // Pattern match to retrieve the value
13 match result {
14     // The division was valid
15     Some(x) => println!("Result: {x}"),
16     // The division was invalid
17     None    => println!("Cannot divide by 0"),
18 }
```



Rust

3.2.12 Option 的典型用途

- 初始值
- 函数定义域不是全集
- 表示简单的错误情况
- 结构体的可选域或者可拿走的域
- 可选的函数参数
- 空指针

3.3 错误处理

3.3.1 错误处理的演进：C 语言错误码

- C 语言：错误码，返回值表示错误
 - 需要检查每个函数调用的返回值
 - 错误处理代码分散在各处，难以维护

```
1  #define DIVISION_BY_ZERO -1
2  #define SUCCESS 0
3  int div(double n, double d, double *r) {
4      if (d == 0) {
5          return DIVISION_BY_ZERO;
6      }
7      *r = n / d;
8      return SUCCESS;
9  }
```



```
1  int main(){
2      double r;
3      if (div(2.0, 0.0, &r) == SUCCESS) {
4          printf("Result: %f\n", r);
5      } else {
6          fprintf(stderr, "Error: Division by zero\n");
7      }
8      return 0;
9  }
```



3.3.3 错误处理的演进：C++ 异常处理

- C++：异常处理，try/catch
 - 异常可以跨函数边界传播
 - 需要手动抛出异常和手动处理，且异常抛出时需要栈回退（stack unwinding）机制，会带来额外性能开销

3.3.4 C++ 异常处理示例

3.3 错误处理

```
1  double divide(double n, double d) {
2      if (d == 0.0) {
3          throw std::runtime_error("Error!");
4      }
5      return n / d;
6  }
7  int main() {
8      try {
9          double r = divide(2.0, 0.0);
10         std::cout << "Result: " << r << '\n';
11     } catch (const std::runtime_error& e) {
12         std::cerr << "Error: " << e.what() << '\n';
13     }
14 }
```



3.3.5 Rust 的错误处理机制：便捷和效率的权衡

3.3 错误处理

```
1  fn divide(n: f64, d: f64) -> Result<f64, String> {
2      if d == 0.0 {
3          return Err(String::from("Cannot divide by zero"));
4      }
5      Ok(n / d)
6  }
7  fn main() {
8      match divide(2.0, 0.0) {
9          Ok(r) => println!("Result: {}", r),
10         Err(e) => eprintln!("Error: {}", e),
11     }
12 }
```



3.3.6 Rust 的错误处理机制

- 对于不可恢复的错误，使用恐慌 `panic!`。
 - 数组越界、栈越界、算术运算溢出.....
- 对于可恢复的错误，使用 `Result`。
 - 文件操作、网络操作、字符串解析.....


```
1 enum Result<T, E> {  
2     Ok(T),  
3     Err(E)  
4 }  
5  
6 use Result::{Ok, Err};
```



- Result 与 Option 类似，除了正常结果外，还可以表示错误状态。
- 也定义了 unwrap 和 expect 等方法。
- 可以通过 ok 或 err 等方法转换成 Option。
 - 把 Ok 或者 Err 的值作为 Some，另一种变成 None。
- 也可以进行类似 Option 的操作。
 - and、or、.....

3.3.8 Result<T, E> 的处理原则

- 对于返回结果是 Result 的函数，一定要显式进行处理。
 - 可以使用 unwrap/expect，也可以通过匹配合理地处理 Ok/Err 状态。
 - 如果不处理，编译器会发出警告。
 - 不正确处理可能会带来潜在问题，导致意想不到的情况。

3.3.9 自定义 Result 别名

- 一种常见的做法是在自己编写的库里使用自定义的错误类型，并定义 Result 的别名。

```
1 use std::io::Error;  
2  
3 type Result<T> = std::result::Result<T, Error>;
```

 Rust

- 除了固定 E = Error 以外与 std::Result 是等价的。
- 使用的时候要注意名字空间。

```
1 use std::io;  
2  
3 fn foo() -> io::Result<()> {  
4     // ...  
5 }
```

 Rust

3.3.10 ? 操作符

- 配合 Result 类型

```
1 fn read_username_from_file() -> Result<String, io::Error> {  
2     let mut username = String::new();  
3     File::open("hello.txt)?.read_to_string(&mut username)?;  
4     Ok(username)  
5 }
```



- 配合 Option 类型

```
1 fn last_char_of_first_line(text: &str) -> Option<char> {  
2     text.lines().next()?.chars().last()  
3 }
```



3.3.11 ? 操作符的原理

- 作用：提前传播错误
- 场合：返回值是 `Result` 或者 `Option` 函数中
- 对于 `Result` 类型，
 - 如果是 `Err` 则提前返回，当前函数立即返回该错误。
 - 否则，从 `Ok` 中取出返回值作为 `?` 操作符的结果。
- 对于 `Option` 类型，
 - 如果是 `None` 则提前返回，当前函数立即返回 `None`。
 - 否则，从 `Some` 中取出返回值作为 `?` 操作符的结果。

3.3.12 错误处理的原则

- 恐慌，还是不恐慌？关键看是否要给程序提供**恢复的机会**。
- 用 `unwrap/expect` 的场合：
 - `unwrap/expect` 可以作为原型代码中的错误处理占位符。
 - 当用户有更多的信息，能够保证调用不出错。
- 原则：错误发生后程序是否进入一个坏的状态？
 - 坏的状态是指不经常出现的情况，而不是像用户输入不合法这样经常可能出现的情况。
 - 之后的代码会依赖程序不处于坏的状态。
 - 没有办法用目前使用的类型来表示这种信息。

3.4 容器

- 连续空间、可增长的序列，末尾可以高效增删
- 会发生增长和收缩
- 最常用的容器

3.4.2 Vec 的 get_disjoint_mut

- 借用规则不允许依次取得多个可变引用，Rust 1.86 起可以用 `get_disjoint_mut` 同时获得多个**互不相同**元素的可变引用（返回 `Result<[&mut T; N], GetDisjointMutError>`）。
- 也可以用范围同时获得多个不重叠的切片。

```
1 let mut v = vec![1, 2, 3, 4, 5, 6];
2
3 let [a, b] = v.get_disjoint_mut([0, 5]).unwrap();
4 *a += 10;
5 *b += 10;
6 let [s1, s2] = v.get_disjoint_mut([0..2, 3..5]).unwrap();
7 s1[0] += 100; // s1: &mut [i32]
8 s2[1] += 100;
```



3.4.3 VecDeque<T>

- 双端向量，两端可以高效增删
- 用环状缓冲区来实现

- 双向链表
- 不能随机索引

- 映射 / 字典类型
- 一般使用 HashMap<K, V>
 - 需要满足 $K: Hash + Eq$
 - 使用哈希表实现，没有顺序，效率较高， $O(1)$ 的访存复杂度
- 需要有序的时候用 BTreeMap<K, V>
 - 需要满足 $K: Ord$
 - 使用 B 树实现，有序，效率相对低一些， $O(\log n)$ 的访存复杂度

3.4.6 创建新的哈希表

```
1 use std::collections::HashMap;  
2  
3 let mut scores = HashMap::new();  
4  
5 scores.insert(String::from("Blue"), 10);  
6 scores.insert(String::from("Yellow"), 50);
```



3.4.7 访问哈希表的元素

```
1 let mut scores = HashMap::new();  
2  
3 scores.insert(String::from("Blue"), 10);  
4 scores.insert(String::from("Yellow"), 50);  
5  
6 let team_name = String::from("Blue");  
7 let score = scores.get(&team_name);
```



3.4.8 迭代哈希表

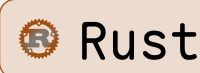
```
1 let mut scores = HashMap::new();
2
3 scores.insert(String::from("Blue"), 10);
4 scores.insert(String::from("Yellow"), 50);
5
6 for (key, value) in &scores {
7     println!("{}", key, value);
8 }
```



3.4.9 哈希表和所有权

- 对于 Copy 类型，拷贝进哈希表。
- 对于非 Copy 类型，移动进哈希表，哈希表拥有所有权。

```
1 let field_name = String::from("Favorite color");  
2 let field_value = String::from("Blue");  
3  
4 let mut map = HashMap::new();  
5 map.insert(field_name, field_value);  
6 // field_name and field_value are invalid at this point, try  
7 // using them and see what compiler error you get!
```



3.4.10 HashMap 的 `get_disjoint_mut`

- 哈希表中的 `get_disjoint_mut` 通过键指定多个元素，键互不不同时可以同时获得它们的可变引用。
- 返回类型是 `[Option<&mut V>; N]`，键不存在时为 `None`。

```
1 let mut scores = HashMap::new();
2 scores.insert(String::from("Blue"), 10);
3 scores.insert(String::from("Yellow"), 50);
4
5 let [blue, yellow] = scores.get_disjoint_mut(["Blue", "Yellow"]);
6 if let Some(b) = blue { *b += 1; }
7 if let Some(y) = yellow { *y += 1; }
```



Rust

3.4.11 更新哈希表

- 改写

```
1 scores.insert(String::from("Blue"), 10);
```



- 不存在时添加

```
1 scores.entry(String::from("Blue")).or_insert(50);
```



- 基于旧值更新

```
1 let text = "hello world wonderful world";
2 let mut map = HashMap::new();
3 for word in text.split_whitespace() {
4     let count = map.entry(word).or_insert(0);
5     *count += 1;
6 }
```



- 集合，元素是唯一的
- HashSet<T> 和 BTreeSet<T> 就是在 HashMap<T, ()> 和 BTreeMap<T, ()> 上包了一层。
- 需求和表现跟相应的 Map 相同。

- 用二叉最大堆实现的优先级队列
- 弹出元素时返回目前堆中的最大值

3.5 迭代器

3.5.1 迭代器的定义

```
1 pub trait Iterator {  
2     type Item;  
3     fn next(&mut self) -> Option<Self::Item>;  
4     // More methods omitted  
5 }
```



- 迭代器特型包含一个相关的类型 `Item`，以及会产生该类型对象的方法 `next`。
- 其他方法有使用 `next` 的默认实现版本。

3.5.2 迭代器与所有权

- 有三种迭代类型：
 - `into_iter()`，产生 `T` 类型。
 - `iter()`，产生 `&T` 类型。
 - `iter_mut()`，产生 `&mut T`。
- 集合可以提供部分或者全部接口。

3.5.3 迭代器与循环

- `for ... in ...` 循环是一种语法糖，可以展开为使用迭代器的循环，例如：

```
1 for i in 0..10 {  
2     println!("{}", i);  
3 }
```

 Rust

可以展开为¹：

```
1 let mut iter = (0..10).into_iter();  
2 while let Some(i) = iter.next() {  
3     println!("{}", i);  
4 }
```

 Rust

¹ 实际上的展开更加复杂，要考虑临时值的生命周期以及循环变量的类型推导等因素，详见[参考手册](#)。

3.5.4 迭代器支持的操作

- 迭代器类型必须支持 `next` 操作，如果自己实现迭代器，必须提供 `next` 方法。
- 其他操作是通过调用 `next` 方法配合其他代码来实现的，有默认的实现版本。
- 操作大致可以分为两类：
 - 汇聚操作，例如 `collect`、`fold`、`any`、`all` 等，调用后会立刻得到结果。
 - 变换操作，例如 `map`、`filter`、`enumerate` 等，是惰性的。

- `collect()` 把（惰性的）迭代器变成一个实际的集合。
- `collect()` 有时候需要提供类型提示来通过编译。
 - 结果可以是任何的集合（或者容器）。

```
1 fn collect<B>(self) -> B where B: FromIterator<Self::Item>
2
3 let vs = vec![1, 2, 3, 4];
4 // Error: What type is this? It cannot infer!
5 let set = vs.iter().collect();
6 // Hint to `collect` that we want a HashSet back.
7 // Note the lack of an explicit <i32>.
8 let set: HashSet<_> = vs.iter().collect();
9 // Alternate syntax! The "turbofish" ::<>
10 let set = vs.iter().collect::
```



```
1 fn fold<B, F>(self, init: B, f: F) -> B
2     where F: FnMut(B, Self::Item) -> B;
3
4 let vs = vec![1, 2, 3, 4, 5];
5 let sum = vs.iter().fold(0, |acc, &x| acc + x);
6 assert_eq!(sum, 15);
```



Rust

- fold 把迭代器折叠成一个单一的值。
 - 在其他语言或者系统里也称为 reduce 或 inject 操作。
- fold 有两个参数：
 - 初始值，或者积累值，B 类型的 acc
 - 一个“折叠”函数，参数是一个 B 类型和一个 Item 类型，返回值是 B 类型。

3.5.7 find 和 position

```
1 fn find<P>(&mut self, predicate: P) ->  
  Option<Self::Item>  
2     where P: FnMut(Self::Item) -> bool;  
3  
4 fn position<P>(&mut self, predicate: P) -> Option<usize>  
5     where P: FnMut(Self::Item) -> bool;
```



- 找出迭代器中第一个满足谓词函数 predicate 的元素。
 - find 返回项目本身。
 - position 返回项目的索引。
- 没找到都返回 None。

```
1 fn any<F>(&mut self, f: F) -> bool
2     where F: FnMut(Self::Item) -> bool;
3
4 fn all<F>(&mut self, f: F) -> bool
5     where F: FnMut(Self::Item) -> bool;
```



- any 测试迭代器中是否存在元素符合给定函数。
- all 测试迭代器中所有元素是否都符合给定函数。
- 逻辑或和逻辑与的关系

3.5.9 迭代器适配器

- 适配器 (adapters) 操作一个迭代器，返回另一个迭代器。
- 适配器通常是惰性的：除非不得不做，不然先不去求值。
- 必须显式使用或者用 `for` 循环迭代才会去求值。

```
1 fn map<B, F>(self, f: F) -> Map<Self, F>
2     where F: FnMut(Self::Item) -> B;
3
4 let vs = vec![1, 2, 3, 4, 5];
5 let twice_vs: Vec<_> = vs.iter().map(|x| x * 2).collect();
```



- map 接受一个函数，创建一个迭代器，在每个元素上调用这个函数。
- 完成从集合 `Collection<A>` 和 `A -> B` 函数得到 `Collection<B>` 的操作。
 - 这里，`Collection` 不是一种实际的类型。

```
1 fn filter<P>(self, predicate: P) -> Filter<Self, P>  
2     where P: FnMut(&Self::Item) -> bool;
```



- filter 接受一个谓词函数 P，把不符合谓词的元素都去掉。
- filter 返回 Filter<Self, P>，需要用 collect 获得集合。

3.5.12 take 和 take_while

```
1 fn take(self, n: usize) -> Take<Self>;  
2  
3 fn take_while<P>(self, predicate: P) -> TakeWhile<Self, P>  
4     where P: FnMut(&Self::Item) -> bool;
```

 Rust

- take 创建一个迭代器，返回前 n 个元素。
- take_while 接受一个谓词，迭代直到谓词返回 false。
- 可以在无限范围上使用得到有限序列：

```
1 for i in (0..).take(5) {  
2     println!("{}", i); // Prints 0 1 2 3 4  
3 }
```

 Rust

```
1 fn skip(self, n: usize) -> Skip<Self>;
```



- 返回一个新的迭代器，跳过前 n 个元素。

```
1 fn enumerate(self) -> Enumerate<Self>;
```



- 用于迭代集合时同时需要序号和元素。
- 返回 (index, value) 的迭代器，index 是 `usize` 类型的序号。

```
1 fn zip<U>(self, other: U) -> Zip<Self, U::IntoIter>  
2     where U: IntoIterator;
```



- 把两个迭代器逐项合并成一个新的迭代器。
- 调用形式：a.iter().zip(b.iter())
 - 返回项目的形式：(ai, bi)
- 当一个输入迭代器结束时，整个 zip 输出的迭代器结束。

```
1 fn cloned<'a, T>(self) -> Cloned<Self>  
2     where T: 'a + Clone, Self: Iterator<Item=&'a T>;
```



- 创建一个迭代器，在每个元素上调用 clone 方法。
- 相当于 `vs.iter().map(|v| v.clone())`。
- 在目前有 `&T` 迭代器，想有 `T` 迭代器的时候使用。

3.6 自动测试

3.6.1 单元测试

- 软件工程中的**规模代价平方定律**
 - 定位并修复一个问题所需的代价正比于目标代码规模的平方
 - 为了减少错误修复的成本，要尽可能早地发现错误，并在尽量小的范围内定位并修复错误
- **单元测试**是针对程序模块（软件设计的最小单位）来进行正确性检验的测试工作

3.6.2 测试函数

单元测试是很常见且必要的功能，因此 Rust 自带了测试的支持。

Rust 中用函数来实现单元测试，函数前面加上 `#[test]` 标注这是一个测试函数：

```
1  #[test]
2  fn it_works() {
3      let result = 2 + 2;
4      assert_eq!(result, 4);
5  }
```



3.6.3 运行测试

为了运行项目中的所有测试函数，可以用 `cargo test` 命令：

```
1 $ cargo test
2 running 1 test
3 test tests::it_works ... ok
4
5 test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured;
  0 filtered out;
6 finished in 0.00s
```

 Shell

它会汇报每个测试的结果，并最后给出所有测试通过情况的统计信息。

3.6.4 编写测试

为了确认程序行为符合预期，一般可以用如下的方式进行检查：

- `assert!()` 宏，断言一个值一定是 `true`
- `assert_eq!()` 宏，断言两个值一定相等
- `Option::unwrap` 或 `Option::expect`，断言一个 `Option` 类型的值一定是 `Some(_)`

```
1  #[test]
2  fn it_works() {
3      let result = 2 + 2;
4      assert_eq!(result, 4);
5  }
```



在测试函数中出现的恐慌会被捕捉下来，并标记当前测试失败，而不会退出程序。

3.6.5 测试例子

通常会把单元测试与被测试的代码放在同一个文件中：

```
1 fn vector_length(data: &Vec<i32>) -> usize {  
2     data.len()  
3 }  
4  
5 #[test]  
6 fn test_vector_length() {  
7     assert_eq!(vector_length(&vec![1, 2, 3]), 3);  
8 }
```



养成良好的习惯：每写一个函数，就在同一个文件中实现对它的单元测试。

3.6.6 测试例子（子模块）

有时候会希望测试代码仅在测试模式下编译，这样不会拖累正常编译的时间。此时可以使用 `#[cfg(test)]` 实现条件编译：

```
1  fn vector_length(data: &Vec<i32>) -> usize { data.len() }
2
3  #[cfg(test)]
4  mod tests {
5      use super::vector_length;
6
7      #[test]
8      fn test_vector_length() {
9          assert_eq!(vector_length(&vec![1, 2, 3]), 3);
10     }
11 }
```



3.6.7 更多测试选项

默认情况下，`cargo test` 会同时运行多个测试以加快速度。如果你的测试在同时运行的情况下可能出错，或者希望它按顺序执行，那么可以使用 `cargo test -- --test-threads=1` 命令来指定它同时只进行一个测试。

如果想要指定只运行单个测试，可以使用 `cargo test -- test_2` 命令来仅运行 `test_2` 这个测试：

```
1  #[test]
2  fn test_1() { }
3
4  #[test]
5  fn test_2() { }
```



3.7 小结

3.7.1 本讲小结

- 字符串
- Option 枚举类型
- 错误处理：是否恐慌？
- 容器和迭代器
- 自动测试

下一讲：泛型、特型和生命周期